UFUG2106 Final Project —Round 5 阶段性本地复现实验报告

Round: Round 5 —BDD / Implicant Reuse and Efficient Netlist Evaluation
实验日期： 2026-11-11 13:05（北京时间）
本地证据： `20261111_130515_Round5_local_results.zip`
状态：已完成本地配对复现，可作为 Round 5 阶段性技术记录；未声称隐藏测试集或最终课程评分。

1. 研究目的与关键结论

Round 5 延续 Round 4.1 Balanced 的布尔逻辑综合框架。我们希望优化的是**综合器的计算效率**，不牺牲已经验证的 Area、Delay 和功能正确性。针对重复 BDD Construction、重复 Implicant Cover Enumeration 和反复文本解析的 Netlist Evaluation，Round 5 使用共享只读结构、符号 Cover 缓存、严格重复候选消除与单次线性扫描评价器。

指标	本次重新编译的 Round 4.1 Balanced	Round 5	变化
Official PASS	25/25	25/25	不变
Target Delay Met	25/25	25/25	不变
25 案例 Area 总和	529,014	529,014	**不变**
Median Runtime	87.041 ms	**68.861 ms**	**−20.89%**
Maximum Runtime	2477.652 ms	**1787.182 ms**	−27.87%
Runtime 总和	8695.436 ms	**6030.508 ms**	**−30.65%**
输出 Netlist 逐字节相同	—	**25/25**	全部一致

两版在**同一次 WSL 运行中分别从源码编译**，因此这里的配对比较比跨机器容器测试更具可比性，但仍是一次测量，不足以估计性能波动范围或统计显著性。

统计口径澄清： Median Runtime 的下降比例是两组中位数之比，而 Runtime 总和的下降比例是两组总耗时之比。这两者回答不同问题，不能混作同一个“提速率”。

2. 实验环境、输入和完整性

- **OS / Kernel:** Ubuntu 24.04 under WSL2, Linux `6.6.87.2-microsoft-standard-WSL2`。
- **CPU:** Intel Core Ultra 9 275HX；WSL 报告 24 个逻辑 CPU。
- **Compiler:** GCC 13.3.0，C++17；Python 3.12.3。
- **输入：**教师提供的 25 个公开 Stage 1 Truth Tables，实验同时保留逐例哈希。
- **Evaluator：**教师提供的 Linux Evaluator，SHA-256：`142361d7ce18`⋯`dbe7`（完整 64 位摘要保留在 `environment.txt`）。
- **源代码：**本地实验附有 `source_snapshot/`；其 SHA-256 与 `environment.txt` 所列记录吻合。
- **证据形式：**官方 CSV / Log、两套 Netlist、逐案例对比、独立验证、Profiling、110 生成测试及 2 项压力测试。

3. Round 5 算法设计及数学依据

3.1 固定变量顺序的 ROBDD 构造缓存

令每个输出为布尔函数 $f_j : \{0,1\}^n \to \{0,1\}$，变量顺序为排列 π。其 Shannon 分解恒等式是

$$f_j = (\neg x_{\pi(k)} \wedge (f_j)_{x_{\pi(k)}=0}) \vee (x_{\pi(k)} \wedge (f_j)_{x_{\pi(k)}=1}).$$

经同构节点合并和冗余决策消除得到固定顺序的 ROBDD。给定同一 Truth Table、变量顺序、节点正规化规则和唯一表，构造出的只读 BDD 图与后续的 Mapping 参数无关。

记构造为 $G = \text{BuildBDD}(f, \pi)$，不同 Mapping 配置为 $\theta_1, \dots, \theta_k$。原先的重复计算可改为

$$\text{Map}(\text{BuildBDD}(f, \pi), \theta_i) \;\longrightarrow\; \text{Map}(G, \theta_i), \quad i = 1, \dots, k.$$

只要缓存是不可变的、不会跨不同 Truth Table 或变量顺序错误复用，并且各 Mapper 拥有独立的可变门表，则**某个固定候选的 Boolean Function、结构和评价值不发生改变**。

设一次 BDD 构造耗时为 T_B，第 i 个映射候选耗时为 $T_{M,i}$，则仅对同一变量顺序下的 k 个候选，有

$$T_{\text{old}} \approx kT_B + \sum_{i=1}^{k} T_{M,i}, \qquad T_{\text{cached}} \approx T_B + \sum_{i=1}^{k} T_{M,i}.$$

节省的主要是 $(k-1)T_B$，不是整个搜索时间的 $(k-1)/k$。缓存需要额外驻留内存；当前实现限制为 $n \le 18$ 且 BDD 节点数不超过 80,000。

3.2 Prime Implicant / Greedy Cover 缓存

对小支持集输出，SOP/POS 方法先计算函数的 essential support，再生成覆盖 ON-set 或 OFF-set 的合法 Cubes，并用确定性的 Greedy Cover 选择一组 Implicants。**Symbolic Cover 只依赖输入函数、极性与既定贪心规则，不依赖后续门级 Mapping 的 Gate Costs**。因此可缓存支持集、选出的 Cubes 与状态，再分别映射到 AND2/OR2/NOT 电路。

注意，Cover 等价性来自它覆盖全部目标赋值、且不覆盖错误赋值的性质，不意味着贪心 Cover 面积最优。缓存也不会降低 Prime Implicant Enumeration 的最坏指数复杂度。当前受原有阈值保护（small support，且仅在 $n \le 13$ 时启用）。

3.3 删除严格重复的候选

原配置中，有一组候选除 `cubemode` 外其他参数相同；但当 $n > 13$ 时，Implicant 分支被禁用，因此它们事实上产生相同的固定候选。Round 5 仅在这个可证明等价的条件下跳过重复尝试。此操作节约计算，但不排除墙钟截止时间变化会改变后续完成候选的数量。

3.4 更快的 Netlist Area / Delay 评价

对合法的拓扑有序 Netlist，设置输入和常量 0 的 Arrival Time 为 0，门节点按以下递推计算：

$$T(v) = \begin{cases} T(a) + d_{\text{NOT}}, & v = \neg a, \\ \max\{T(a), T(b)\} + d_{\text{AND}}, & v = a \wedge b, \\ \max\{T(a), T(b)\} + d_{\text{OR}}, & v = a \vee b. \end{cases}$$

于是 $D(C) = \max_j T(o_j)$，$A(C) = \sum_{g \in C} a(g)$。新代码用轻量指针扫描解析自身生成的合法 Netlist，减少原 `istringstream`、`stoi` 和大量短字符串分配的常数开销。这是**同一成本函数的更快计算**，不是新的逻辑等价变换。

4. 本地性能结果和瓶颈定位

4.1 绝对节省时间最大的案例

Case	Round 4.1 Runtime	Round 5 Runtime	节省
case_024.txt	2,477.652 ms	**1,787.182 ms**	690.470 ms (27.87%)
case_025.txt	1,234.425 ms	**790.057 ms**	444.368 ms (36.00%)
case_001.txt	847.895 ms	**557.764 ms**	290.131 ms (34.22%)
case_007.txt	1,030.063 ms	**762.796 ms**	267.267 ms (25.95%)
case_015.txt	565.101 ms	**366.987 ms**	198.114 ms (35.06%)
case_021.txt	567.564 ms	**420.715 ms**	146.849 ms (25.87%)
case_014.txt	376.718 ms	**242.159 ms**	134.559 ms (35.72%)

以上全部案例的 Area 和 Delay 都没有变化。`case_024` 仍是最大 Runtime 的案例，后续研究应着重关注它的 Mapping 和后处理成本。

4.2 Profiling 所揭示的开销

采样 Profiling 以各阶段日志为准，属于**额外单独执行的带仪表测试**，不能与官方 Evaluator Runtime 逐毫秒相加。

Case	BDD 构造合计	Gate Mapping 合计	BDD Cache Hits / Calls
case_001	10.79 ms	199.53 ms	2/7
case_005	0.03 ms	11.35 ms	3/5
case_024	62.79 ms	917.46 ms	2/7

例如 `case_024` 的记录中，BDD 构造合计仅约 62.8 ms，而 Mapping 合计约 917.5 ms。可见 BDD 缓存确实消除了重复构造，但**不能单独解释整体提速**；后续最重要的优化应针对 Mapping、Area Recovery 和 Netlist 内存表示。

4.3 全部 25 个公开案例的逐例对照

Case	Area（两版相同）	Delay（两版相同）	Round 4.1 (ms)	Round 5 (ms)	降幅
001	61,952	62	847.895	557.764	34.22%
002	18,897	60	299.460	203.947	31.90%
003	24,251	56	334.859	227.636	32.02%
004	4,550	47	32.896	23.466	28.67%
005	700	31	43.081	26.091	39.44%
006	5,962	46	41.987	28.630	31.81%
007	78,042	71	1030.063	762.796	25.95%
008	8,637	50	53.351	38.939	27.01%
009	14,584	46	87.041	68.861	20.89%
010	4,712	32	142.901	121.846	14.73%
011	464	33	51.879	29.868	42.43%
012	5,359	25	41.385	31.236	24.52%
013	11,252	60	141.519	101.089	28.57%
014	11,662	61	376.718	242.159	35.72%
015	33,374	64	565.101	366.987	35.06%

Case	Area（两版相同）	Delay（两版相同）	Round 4.1 (ms)	Round 5 (ms)	降幅
016	2,149	40	21.324	16.432	22.94%
017	440	22	9.887	6.781	31.41%
018	1,232	31	18.215	12.252	32.74%
019	5,319	57	166.373	101.077	39.25%
020	766	30	42.161	25.068	40.54%
021	28,104	60	567.564	420.715	25.87%
022	1,320	31	54.687	31.379	42.62%
023	125	22	13.012	8.250	36.60%
024	124,475	70	2477.652	1787.182	27.87%
025	80,686	60	1234.425	790.057	36.00%

5. 功能验证、泛化测试与边界

- **公开 Stage 1：**官方 Evaluator 分别确认两版 25/25 PASS 且 25/25 Target Met。
- **输出一致性：**两组 `outputs/` 共 25 对文件均经字节级比较确认完全一致。
- **独立正确性验证：**25/25 成功；其中 13 个案例穷举全部输入，12 个大规模案例进行确定性抽样，Area/Delay 与官方结果相符。
- **额外生成案例：**110/110 PASS；其中 96/110 满足各自生成的 Target Delay。其余 Target 不达标不等于功能失败，也不能推断目标在数学上不可行。
- **压力测试：**`case_901`（n=20, m=8）PASS，253.858 ms；`case_902`（n=16, m=4）PASS，1,506.674 ms，均满足各自目标。

重要局限：固定候选结构的缓存等价性**不代表时间截断的整个综合程序在任意环境下都输出同一 Netlist**。改变运行速度可能让搜索在截止时间前完成不同候选。我们这次观察到 25/25 完全一致，但它是实验事实，不是隐藏测试的普遍定理。另外，单个不可中断的 Gate Mapping 或后处理步骤仍可能超出严格时间预算。

6. 结论与下一阶段建议

本地实测 Round 5 在保持全部 25 个电路逐字节相同、Area 总和 529,014 不变的情况下，将 Median Runtime 从 87.041 ms 降到 68.861 ms（降低 20.89%），总 Runtime 降低 30.65%。这证明 **Engineering Optimization 可以在不牺牲电路质量的情况下显著提升综合效率**。

研究上的下一步应优先考虑：**(1)** 使用类型化 In-memory Netlist，避免 Mapping、Balancing、Polarity Recovery 之间的文本序列化；**(2)** 对引用计数与局部 Critical Path 做增量 Cost Evaluation；**(3)** 在有明确单候选超时/内存保护之后，再尝试更丰富的 Cut-based Rewriting。

建议把当前 Round 5 作为新的 **Efficiency Baseline**，同时保留 Round 4.1 Balanced 为 Frozen Comparator。正式总 Report 中应将这轮表述为 `Semantics-preserving computational optimization with time-budget caveats`，而非新 Boolean Function Simplification。

7. 证据清单与可复现性

源数据来自上传的本地 ZIP：`reference/official.csv`、`round5/official.csv`、两套 `outputs/`、`round5/independent_verifica`
`round5/generated_110.csv`、`round5/stress.csv`、`profiles/`、`environment.txt`、`source_snapshot/`、`per_case_comparison.csv` 和 `summary.json`。本报告附带标准化 `Round5_Local_PerCase_Comparison.csv`；原始结果包应长期保留作为不可变实验原始证据。